CHULETA

pandas en dos páginas

Las operaciones que más vas a usar para pasar de un CSV a una conclusión. Ejemplos con **ventas.csv**: columnas `id, fecha, mes, ciudad, ventas`.

Flujo típico: Cargar → Explorar → Limpiar → Transformar → Agrupar → Graficar

1 Importar y leer

```python
import pandas as pd

df = pd.read_csv("ventas.csv")
# Opciones útiles
df = pd.read_csv("ventas.csv",
    sep=";", decimal=",",          # CSV en formato latino
    encoding="utf-8",
    parse_dates=["fecha"])
```

Lee un archivo CSV y devuelve un **DataFrame**: una tabla con filas y columnas con nombre.

2 Explorar

```python
df.head()          # primeras 5 filas
df.tail(3)         # últimas 3 filas
df.shape           # (filas, columnas)
df.info()          # tipos y valores no nulos
df.describe()      # estadísticas numéricas
df["ciudad"].value_counts()
```

Lo primero siempre: mira los datos, sus **tipos** y cuántos valores **faltan**.

3 Seleccionar columnas y filas

```python
df["ventas"]                # una columna (Series)
df[["ciudad", "ventas"]]    # varias columnas
df.loc[0, "ciudad"]         # por etiqueta
df.iloc[0:5, 0:2]           # por posición
```

`loc` usa nombres; `iloc` usa posiciones numéricas.

4 Filtrar

```python
df[df["ventas"] > 1000]
df[(df["ciudad"] == "Cali") & (df["mes"] == 3)]
df[df["ciudad"].isin(["Bogotá", "Medellín"])]
df.query("ventas > 1000 and mes <= 6")
```

Combina condiciones con `&` (y) y `|` (o), cada una **entre paréntesis**.

5 Limpiar

```python
df.isna().sum()                 # nulos por columna
df = df.dropna(subset=["ventas"])
df["ciudad"] = df["ciudad"].str.strip()
df["ventas"] = df["ventas"].astype(float)
df = df.drop_duplicates()
df = df.rename(columns={"vta": "ventas"})
```

Casi todas las operaciones devuelven un DataFrame **nuevo**: asígnalo de vuelta.

6 Crear columnas y ordenar

```python
df["ventas_usd"] = df["ventas"] / 4000
df["trimestre"] = df["fecha"].dt.quarter
df.sort_values("ventas", ascending=False)
```

Las operaciones se aplican a **toda la columna** a la vez, sin bucles.

7 Trabajar con fechas

```python
df["fecha"] = pd.to_datetime(df["fecha"],
                             dayfirst=True)
df["anio"] = df["fecha"].dt.year
df[df["fecha"] >= "2025-07-01"]
df.resample("ME", on="fecha")["ventas"].sum()
```

Con `dayfirst=True`, «03/04/2025» se lee como 3 de abril.

8 Trabajar con texto

```python
df["ciudad"].str.lower()
df["ciudad"].str.contains("ba", case=False)
df["codigo"].str.split("-", expand=True)
df["ciudad"].replace({"Bogota": "Bogotá"})
```

Los métodos de texto se usan a través de `.str` y omiten los nulos.

9 Agrupar con groupby

```
df.groupby("ciudad")["ventas"].sum()

(df.groupby(["ciudad", "mes"])
   .agg(total=("ventas", "sum"),
        promedio=("ventas", "mean"),
        pedidos=("id", "count"))
   .reset_index())
```

Patrón **dividir → aplicar → combinar**: separa por grupo, calcula y une los resultados.

10 Combinar tablas con merge

```
ciudades = pd.read_csv("ciudades.csv")
# columnas: ciudad, departamento, poblacion

df2 = df.merge(ciudades, on="ciudad",
               how="left")
```

Une dos tablas por una columna en común, como un **JOIN** de SQL.

- **how="inner"** Solo las claves que están en ambas tablas.
- **how="left"** Todas las filas de la izquierda; NaN si no hay pareja.
- **how="right"** Todas las filas de la derecha.
- **how="outer"** Todas las filas de ambas tablas.

11 Graficar

```
import matplotlib.pyplot as plt

totales = df.groupby("ciudad")["ventas"].sum()
totales.sort_values().plot.barh(
    title="Ventas por ciudad")
plt.xlabel("Millones de COP")
plt.tight_layout()
plt.savefig("ventas.png", dpi=150)
```

Otros tipos: `.plot.line()`, `.plot.hist()`, `.plot.scatter(x=, y=)`.

12 Guardar

```
df.to_csv("limpio.csv", index=False)
# A Excel (requiere el paquete openpyxl)
df.to_excel("reporte.xlsx", index=False)
```

`index=False` evita guardar el índice como una columna extra.

AGREGACIÓN	QUÉ CALCULA
`sum()`	Suma
`mean() / median()`	Promedio / mediana
`count()`	Valores no nulos
`nunique()`	Valores distintos
`min() / max()`	Mínimo / máximo
`std()`	Desviación estándar

TIPO (DTYPE)	CONTIENE
`int64`	Enteros
`float64`	Decimales (y NaN)
`object`	Texto o mezcla
`datetime64`	Fechas: .dt.year, .dt.month
`bool`	Verdadero / falso
`category`	Pocos valores repetidos

13 Ejemplo completo: del CSV a la gráfica

```
import pandas as pd

ventas = (
    pd.read_csv("ventas.csv", parse_dates=["fecha"])
      .dropna(subset=["ventas"])
      .assign(ciudad=lambda d: d["ciudad"].str.strip())
)
resumen = (ventas.groupby("ciudad")["ventas"]
                 .agg(["sum", "mean", "count"])
                 .sort_values("sum", ascending=False))
print(resumen.head())
resumen["sum"].plot.bar(title="Ventas por ciudad", ylabel="Millones de COP")
```

Lee, limpia, resume y grafica en pocas líneas. El resultado es la gráfica de barras de ventas por ciudad del curso.

Consejos rápidos

- Encadena pasos entre paréntesis, un método por línea: es más fácil de leer y de depurar.
- Si ves `SettingWithCopyWarning`, usa `df.loc[filtro, "col"] = valor` o trabaja sobre `df.copy()`.
- Revisa `df.dtypes` después de leer: una columna numérica con un solo texto se carga como `object`.
- Para archivos grandes, lee solo lo necesario con `usecols=[...]` o en partes con `chunksize=`.